



Autonomous robot vacuum system composed of a cleaner robot and a dust storage robot

Jonghoek Kim

System Engineering Department at Sejong University, Seoul-city, South Korea

ARTICLE INFO

Keywords:

Autonomous robot vacuum system
Robot vacuum
Vacuum cleaner
Dust bin
Suction power
Cleaner robot
Dust storage robot
Tethered robots

ABSTRACT

In this paper, we address an autonomous robot vacuum system, composed of two robots. Conventional robot vacuum has a small dust bin, and its suction power is low. This low cleaning power has been considered as a defect of a conventional robot vacuum. If we use a large robot vacuum with high cleaning power, then we cannot clear various narrow environments, such as narrow space under a sofa. To resolve this problem, we address an autonomous robot vacuum system composed of a small Cleaner Robot (CR) and a large dust Storage Robot (SR). The CR and the SR are connected using a hose, thus they must maneuver as a pair. The role of the small CR is to clean various environments, such as narrow space under a sofa. The CR is small, thus it cannot contain a large motor for increasing its cleaning power. The dust sucked by the CR is transmitted to the SR through the hose. The SR is large, hence it can have a large motor for increasing its cleaning power compared to a conventional robot vacuum. To the best of our knowledge, our paper is novel in addressing an autonomous robot vacuum system composed of a CR and a SR. Our paper is unique in addressing the path plan of these two connected robots, so that the entire open workspace can be cleaned by the CR as fast as possible. Under MATLAB simulations, we demonstrate the effectiveness of path planning for our autonomous robot vacuum system. We show that the proposed tethered robots improve the cleaning efficiency, compared to the case where a single large robot vacuum is applied.

1. Introduction

This article introduces an autonomous robot vacuum system. Conventional robot vacuum has a small disk (circular) shape and can maneuver to clean a narrow environment [1]. This small robot vacuum makes it viable to clean even the narrow space under a sofa. However, conventional robot vacuum has a small dust bin, and its suction power is low.

In the case where a conventional robot vacuum is done cleaning, it can return to the base. Then, the base empties the robot's dust bin. The base can be large, so it can store a large amount of dust. However, since a conventional robot vacuum is small, its cleaning power is lower than a vacuum cleaner utilized by human operators. Conventional robot vacuum can only store a small amount of dust, and its cleaning power is lower than a vacuum cleaner utilized by human operators. This low cleaning power has been considered as a defect of a conventional robot vacuum. If we use a large robot vacuum with high cleaning power, then we cannot clear various narrow environments, such as narrow space under a sofa.

In order to resolve this defect, this article proposes an autonomous robot vacuum system composed of a Cleaner Robot (CR) and a dust Storage Robot (SR). The CR and the SR are connected using a hose, thus they move as a pair.

The proposed vacuum cleaner system takes the concept of the human operated vacuum cleaner, but without the holding bar. Note that a holding bar is not required, since this paper considers an autonomous vacuum cleaner system without human intervention.

E-mail address: jonghoek@sejong.ac.kr.

<https://doi.org/10.1016/j.jfranklin.2024.106938>

Received 25 June 2022; Received in revised form 17 August 2023; Accepted 18 May 2024

Available online 1 June 2024

0016-0032/© 2024 The Franklin Institute. Published by Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



Fig. 1. A vacuum cleaner utilized by human operators. Conventional robot vacuum can only store a small amount of dust, and its cleaning power is lower than a vacuum cleaner utilized by human operators. To resolve this problem, this paper proposes an autonomous robot vacuum system composed of a CR and a SR. The component inside a small red rectangle can be considered as the CR, and the other is the SR.

See Fig. 1 for an illustration of a vacuum cleaner utilized by human operators. Inside the human operated vacuum cleaner (large black box in Fig. 1), there is a large motor for increasing its cleaning power. In Fig. 1, the component inside the small red rectangle can be considered as the CR, and the other is the SR.

Inspired by the human operated vacuum cleaner, the CR and the SR in our paper are connected using a hose. This paper introduces a small CR (small red rectangle in Fig. 1) whose role is to maneuver, while cleaning the environment. The CR is small, hence it cannot store a large amount of dust. In addition, it cannot contain a large motor for increasing its cleaning power. The dust sucked by the CR is transmitted to the SR through the hose.

The SR (large black box in Fig. 1) is large, hence it can have a large motor for increasing its cleaning power. We assume that the hose length is not too long, since the SR requires more suction power as the hose length increases. In addition, the SR can store a large amount of dust sucked by the CR.

In our paper, the motion command of the CR is calculated by the computer inside the SR and is sent to the CR in real time. In this manner, we can further reduce the size of the CR, making it viable to clean even the narrow space under a sofa.

There are many papers on tethered robots, such that robots form a tethered chain and that the first one is tethered to a fixed point [2–4]. Controlling a chain of tethered mobile robots can be considered kinematically as an open-chain robotic arm where the mobile robots are considered as a revolute joint and the tether is considered as a variable length link, using a prismatic joint [2]. Refs. [3,4] solved the path planning problem for a tethered mobile robot, which is connected to a fixed base by a cable of finite length. In this case, the reachable space of the robot is restricted by the length of the cable and obstacles. However, [2–4] considered the case where the first robot in the chain is tethered to a fixed point. Moreover, [2–4] did not consider how to plan the path of two connected robots so that the entire workspace is covered.

Considering an environment containing polygonal obstacles, [5] addressed the problem of planning motions for a pair of planar robots connected to one another via a cable of limited length. [5] considered the case where the cable may intersect with polygonal obstacles. This intersection leads to more involved planning formalization than the planning of a single robot. Moreover, [5] did not consider how to plan the path of two connected robots so that the entire workspace is covered.

As far as we know, our paper is novel in addressing how to plan the path of two connected robots, so that the entire workspace is covered by the CR. In our paper, the hose (cable in other papers) length is not too long, since the SR requires more suction power as the hose length increases. Therefore, two robots must maneuver as one pair, while staying close to each other. By moving back and forth, the CR cleans the open workspace close to the SR. Under this simple back and forth maneuver, the hose connecting the two robots does not intersect an obstacle. Thus, we do not require involved planning formalization, as in [5]. Furthermore, tangling of the hose can be avoided by back and forth maneuver of the CR.

Coverage path planning is the task of generating a path that covers the entire bounded workspace, while avoiding obstacles. There are many papers on coverage path planning of a single robot, as presented in this paragraph. Ref. [6] introduced a comprehensive literature survey on the coverage path planning problem. Ref. [7] introduced an algorithm for generating a coverage motion plan for a robot over a holed rectilinear polygonal region. The authors of [8] proposed an image processing and model-based path planner that can generate an efficient path for a robot to cover a spill utilizing aerial images. The authors of [9] introduced the boustrophedon cellular decomposition, which is an exact cellular decomposition approach, for the purposes of coverage. Each cell in the boustrophedon is covered with simple back and forth motions. A widely utilized path planning [10–13] is a series of equidistant parallel paths for covering the survey region. This planning is called the lawnmower planning. As far as we know, our paper is unique in addressing the path plan of two connected robots, so that the entire open workspace can be cleaned by the CR as fast as possible.

Multiple robots can be applied for cooperative exploration of an unknown workspace [14–17]. For time efficient multi-robot exploration, [18–20] utilized the concept of *frontier*, which represents the border between an explored region and an unexplored

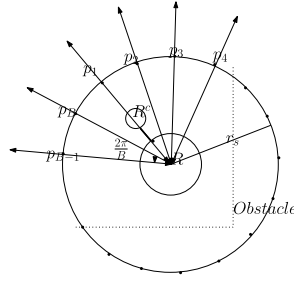


Fig. 2. The sensing boundary centered at R . There is an L-shaped obstacle boundary, marked with dotted line segments. ClearPoints are marked along the sensing boundary. We enumerate all clearPoints in the clockwise direction starting from p_1 .

region. As a robot visits frontiers iteratively, the robot can explore an unknown workspace. The goal of conventional multi-robot exploration is to explore an unknown workspace, considering the case where a robot can be arbitrarily far from another robot.

In our paper, dual-robot system is addressed to cover a given workspace as fast as possible. In our article, the CR and the SR are connected using a hose, thus they must maneuver as a pair. Hence, the problem solved in this paper is different from those solved in other papers on multi-robot exploration.

To the best of our knowledge, our paper is novel in proposing an autonomous robot vacuum system composed of a CR and a SR. Moreover, this article is unique in proposing the path plan of these two connected robots, so that the entire open workspace can be cleaned by the CR as fast as possible.

Under MATLAB simulations, we demonstrate the effectiveness of path planning for our autonomous robot vacuum system. For comparison, we simulate the case where a single large robot vacuum (with high cleaning power) is applied for clearing the workspace. Note that a large robot cannot clear various narrow environments, such as narrow space under a sofa. We show that the proposed tethered robots improve the cleaning efficiency, compared to the case where a single large robot vacuum is applied.

This document is organized as follows: Section 2 introduces assumptions and definitions. Section 3 explains the coverage strategy utilizing two robots that are connected using a hose. Section 4 shows simulation results of the proposed coverage strategy. Section 5 provides Conclusions.

2. Assumptions and definitions

A graph G is a set $G = (V(G), E(G))$, where $V(G)$ is the node set and $E(G)$ is the edge set. Each edge, say $e \in E(G)$ has its associated weight $w(e) : e \rightarrow \mathbb{Z}^+$. Let $\angle(\mathbf{v}_1, \mathbf{v}_2)$ represent the angle formed by two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$. We have $\angle(\mathbf{v}_1, \mathbf{v}_2) = \arccos(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|})$.

Let R present the SR with radius r centered at $\mathbf{R} \in \mathbb{R}^2$. Let R^c present the CR with radius r^c centered at $\mathbf{R}^c \in \mathbb{R}^2$. Both R and R^c are connected using a hose, and the hose is attached at a point, say *AttachPoint*, on R . The hose can extend and shrink, similarly to YOYO. In the case where the hose length is zero, R^c is attached to *AttachPoint* of R directly. Let H present the maximum hose length.

In the case where the hose length is zero, R^c is attached to *AttachPoint* of R directly. This attachment can be performed using the docking approach in modular self-reconfigurable robots [21–23].

The process model of R is $\dot{\mathbf{R}} = \mathbf{v}$. This model is viable, since our article handles a differential wheeled robot, whose movement is based on two separately driven wheels placed on either side of the robot body.

The maximum speed of R is S , i.e., $\|\dot{\mathbf{R}}\| \leq S$. The process model of R^c is $\dot{\mathbf{R}}^c = \mathbf{v}^c$. The maximum speed of R^c is S^c . Note that we do not account for the non-holonomic restrictions to motion, i.e., the turn rate of a robot is not bounded. This assumption is viable, since our document handles a differential wheeled robot.

The SR R detects the boundary of a nearby obstacle utilizing Lidar sensors. The CR R^c can use proximity sensors, such as ultrasound sensors, for detection of a close obstacle.

We assume that R is localized in the indoor environment. For robot localization using Lidar, R can use Lidar simultaneous localization and mapping (SLAM) in [24,25]. We further assume that R can measure the relative position of R^c . Lidar sensors on R can be utilized to measure the relative position of R^c .

While the SR R maneuvers, the CR R^c is attached to *AttachPoint* of R directly (hose length is zero). During the maneuver, we need to make both R and R^c avoid collision with obstacles. Therefore, we say that a *collision occurs* when the distance between $\mathbf{R}$ and any obstacle boundary is less than $rM = r + 2 * r^c$ (safety margin).

A point set $s \in \mathbb{R}^2$, such that $\|\mathbf{R} - s\| = r_s$, is termed the *sensing boundary*. Here, r_s presents the maximum coverage range of R . For example, let us consider an environment in Fig. 2. This figure depicts the sensing boundary centered at R . In this figure, there is an L-shaped obstacle boundary, marked with dotted line segments.

The motion command of R^c is calculated by the computer inside R and is sent to R^c in real time. In this manner, we can further reduce the size of R^c . This is desirable, since the role of R^c is to clean various environments, such as narrow space under a sofa.

In order to make R cover the entire workspace, we let R generate *virtual nodes* in the environment. These virtual nodes are utilized as wayPoints for complete coverage of R . In addition, whenever R generates a virtual node at a point, the CR R^c cleans the open workspace close to the point. Section 3.2.2 explains how to clear the open workspace close to R .

Let v_1 and v_2 present two virtual nodes. Let $\mathbf{v}_1$ and $\mathbf{v}_2$ present the locations of v_1 and v_2 , respectively. Also, $L(\mathbf{v}_1, \mathbf{v}_2)$ presents a straight line segment connecting v_1 and v_2 . Let $\|L(\mathbf{v}_1, \mathbf{v}_2)\|$ present the length of $L(\mathbf{v}_1, \mathbf{v}_2)$.

While the SR maneuvers, the CR R^c is attached to *AttachPoint* of R directly. See Fig. 3. We say that $L(\mathbf{v}_1, \mathbf{v}_2)$ is *safe*, as we satisfy the following case: a circle with radius $rM = r + 2 * r^c$ centered at v_1 can traverse $L(\mathbf{v}_1, \mathbf{v}_2)$, while not intersecting obstacles.

The virtual graph I is a set $I = (V(I), E(I))$, where $V(I)$ is the node set and $E(I)$ is the edge set. Every node in $V(I)$ presents a virtual node. Every edge, say $\{v_1, v_2\} \in E(I)$, implies that $L(\mathbf{v}_1, \mathbf{v}_2)$ is safe and that $\|L(\mathbf{v}_1, \mathbf{v}_2)\| < r_s$. Whenever R generates a new virtual node, I is updated to contain the virtual node. Every edge, say $e \in E(I)$, has its associated weight $w(e) : e \rightarrow Z^+$. The weight of $\{v_1, v_2\}$ indicates $\|L(\mathbf{v}_1, \mathbf{v}_2)\|$.

Based on the definition of $E(I)$, R , which is attached to R^c , does not collide with obstacles as it tracks along an edge in $E(I)$. Hence, I is a safe topological map for attached robots R and R^c . Utilizing the weight of every edge in I , R computes the shortest path between any two virtual nodes. For computing the shortest path in a graph, Dijkstra's algorithms [26] can be applied.

An *open sensing arc* of a virtual node presents an arc of a sensing boundary corresponding to an open workspace without obstacles. A *frontier* of a virtual node is the subset of an open sensing arc, such that all points on the frontier are outside a sensing boundary of every other virtual node. This implies that a frontier is associated to the border between an explored region and an unexplored region. If every virtual node has no frontier, then all virtual nodes cover the entire workspace.

The sensing boundary is discretized as evenly spaced points, termed *discretePoints*. Suppose there are D *discretePoints* on the sensing boundary. Increasing D increases the density of *discretePoints* on the sensing boundary. Among all *discretePoints*, we search for a *discretePoint* meeting the following requirements:

- the *discretePoint* is on a frontier.
- the straight line segment connecting n and the *discretePoint* does not intersect an obstacle.

The found *discretePoint* is termed the *discreteFrontierPoint* of n . Let $f(n)$ present the *discreteFrontierPoint* of n .

The SR utilizes a *discreteFrontierPoint* as a “wayPoint” for clearing the entire workspace. The straight line segment connecting n and a point in $f(n)$ does not intersect an obstacle. Therefore, R , which is attached to R^c , does not collide with obstacles as it maneuvers from n to any point in $f(n)$.

3. Coverage strategy utilizing two connected robots

3.1. Coverage path plan

In our article, one aims to clear an unknown bounded workspace, as fast as possible utilizing the CR and the SR. While two robots maneuver, they need to avoid collision with obstacles. To achieve this, we address the coverage strategy, so that two connected robots explore an unknown bounded workspace while generating a virtual graph in the workspace.

We discuss the proposed coverage path plan strategy in Algs. 1 and 2. Whenever the SR detects no virtual node inside the sensing boundary, it generates a new virtual node. R iteratively searches for the nearest *discreteFrontierPoint*, say f_R , utilizing the virtual graph I . R searches for the shortest path in I from its location to every virtual node with a *discreteFrontierPoint*. Among these virtual nodes with a *discreteFrontierPoint*, R selects the one with the smallest shortest-path. For computing the shortest paths between nodes in I , Dijkstra's algorithms [26] can be applied.

In Alg. 2, the SR tracks along the shortest path in I to the nearest *discreteFrontierPoint* f_R until it reaches f_R . While R maneuvers for arriving at f_R , R^c is attached to *AttachPoint* of R . This implies that the hose length is zero.

We present the path planning method of R for reaching f_R . Suppose R plans to visit virtual nodes $n_1 \rightarrow n_2 \rightarrow \dots \rightarrow n_{end}$ in this order. Here, n_{end} is the virtual node that has f_R as its *discreteFrontierPoint*.

Using the definition of I , this path is safe for R . The length of every line segment along this path is shorter than r_s . Whenever R meets n_j where $j \leq end - 1$, R maneuvers to reach n_{j+1} . Iterate this until R meets n_{end} . After R meets n_{end} , R maneuvers to reach f_R . Note that as R maneuvers, R localizes itself.

Once R reaches f_R , it generates another virtual node. Since a new virtual node is generated, several *discreteFrontierPoints* inside the maximum sensing range of the virtual node are removed. R then searches for the nearest *discreteFrontierPoint* in I , by applying Dijkstra's algorithms [26]. R iterates this until it cannot detect any other *discreteFrontierPoint* by accessing I .

Fig. 3 illustrates the SR deploying a new virtual node. In Fig. 3, both R and R^c are presented as circular robots. See that R^c is attached to *AttachPoint* of R , while R maneuvers. The obstacles are depicted with red curves, and the trajectory of R is presented with a yellow curve. The large dots on the trajectory of R indicate the virtual nodes generated by R . The sensing boundary of each virtual node is depicted as a dotted circle. A frontier of the recently generated sensor is presented with a green curve. In addition, two *discreteFrontierPoints* are presented with two crosses on the frontier.

The following theorem proves that in the case where R cannot detect any frontier, the entire open workspace is covered by sensing boundaries. Theorem 1 proves that R avoids collision, while R maneuvers under Alg. 1.

Theorem 1. *In the case where R cannot detect any frontier, the entire open workspace is covered by sensing boundaries. While R maneuvers under Alg. 1, it avoids collision with obstacles.*

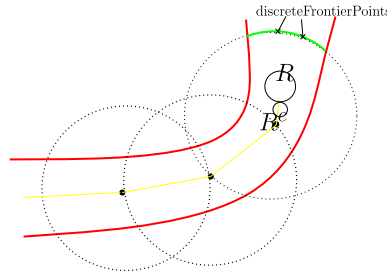


Fig. 3. R deploying a new virtual node. In this figure, both R and R^c are presented as circular robots. See that R^c is attached to *AttachPoint* of R , while R maneuvers. The obstacles are depicted with red curves, and the trajectory of R is presented with a yellow curve. The large dots on the trajectory of R indicate the virtual nodes generated by R . The sensing boundary of each virtual node is depicted as a dotted circle. A frontier of the recently generated sensor is presented with a green curve. In addition, two *discreteFrontierPoints* are presented with two crosses on the frontier. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Proof. Under the transposition rule, the following statement is proved: if unexplored open workspace exists, then R can detect a frontier associated to the unexplored open workspace. Suppose an unexplored open workspace exists. Recall that a frontier of a virtual node is the subset of an open sensing arc, such that all points on the frontier are outside a sensing boundary of every other virtual node. Using the definition of a frontier, at least one virtual node, say n , has a frontier along the boundary of this unexplored open workspace. Therefore, R can detect this frontier.

We next prove that R avoids collision while it maneuvers under Alg. 1. In Alg. 1, R tracks along the shortest path in I to reach f_R . We prove that collision avoidance is assured while R maneuvers until meeting f_R . Let $\{n_1 \rightarrow n_2 \rightarrow \dots \rightarrow n_{end}\}$ present the order of virtual nodes along the shortest path to reach f_R . After meeting n_j ($j \leq end - 1$), R maneuvers until meeting n_{j+1} . Using the definition of $E(I)$, this path is a safe path. In addition, after meeting n_{end} , R maneuvers until meeting f_R . Using the definition of a *discreteFrontierPoint*, R , which is attached to R^c , avoids collision with obstacles while it maneuvers from n_{end} to f_R . $\square$

Algorithm 1 Coverage path plan strategy

```

repeat
  if The SR detects no virtual node inside the sensing boundary then
    The SR  $R$  generates a virtual node at its location;
    By moving back and forth, the CR  $R^c$  clears the open workspace close to  $R$ ;
  end if
  if  $f_R$  is empty then
    Find DiscreteFrontierPoint( $R$ );
  end if
  if The SR  $R$  meets  $f_R$  then
    The SR  $R$  generates a virtual node at its location;
    By moving back and forth,  $R^c$  clears the open workspace close to  $R$ ;
    Find DiscreteFrontierPoint( $R$ );
  else
     $R^c$  is attached to AttachPoint of  $R$  (hose length is zero);
    The SR  $R$  maneuvers until meeting  $f_R$ ;
  end if
until there is no discreteFrontierPoint for every virtual node;

```

Algorithm 2 Find *DiscreteFrontierPoint*(R)

```

The SR searches for the nearest discreteFrontierPoint in  $I$ , by applying Dijkstra's algorithms [26];
The nearest discreteFrontierPoint is set as  $f_R$ ;
The SR  $R$  computes the shortest path in  $I$  to reach  $f_R$ , by applying Dijkstra's algorithms;
The SR  $R$  tracks along the shortest path in  $I$  to reach  $f_R$ ;
While the SR  $R$  maneuvers until reaching  $f_R$ , the CR  $R^c$  is attached to AttachPoint of  $R$ ;

```

3.2. The control law of a robot

3.2.1. The control law of the SR

In this subsection, we introduce control law of the SR. We address the control law for R in discrete-time systems. For convenience, we use $A(k)$ to represent A at sampling-index k . Let T present the sampling interval of our control law.

Suppose $\mathbf{W}$ is the next wayPoint that R will visit along the shortest path to f_R . This implies that R maneuvers to reach $\mathbf{W}$.

Let $\mathbf{v}(k)$ present the velocity of R at sampling-index k . The process model of R is

$$\mathbf{R}(k+1) = \mathbf{R}(k) + \mathbf{v}(k) * T = \mathbf{R}(k) + \eta * \mathbf{g}(k) * T \quad (1)$$

where $\mathbf{g}(k) = \mathbf{W} - \mathbf{R}(k)$. In (1), $\eta > 0$ is a tuning parameter. (1) implies that R maneuvers for reaching $\mathbf{W}$, with a speed proportional to $\|\mathbf{g}(k)\|$.

The maximum speed of R is less than S . Hence, we need to handle the case where $\eta * \|\mathbf{g}(k)\|$ in (1) exceeds S . In the case where $\eta * \|\mathbf{g}(k)\| > S$, (1) is replaced by

$$\mathbf{R}(k+1) = \mathbf{R}(k) + \mathbf{v}(k)T = \mathbf{R}(k) + S \frac{\mathbf{g}(k)}{\|\mathbf{g}(k)\|} T. \quad (2)$$

3.2.2. The control law of the CR

Whenever the SR R generates a virtual node in Alg. 1, the CR R^c cleans the open workspace close to R . We address how R^c cleans the open workspace close to R .

The sensing boundary is discretized as evenly spaced points, called *clearPoints*. Suppose there are B clearPoints on the sensing boundary. Here, $B > 0$ is a tuning constant. As B increases, we increase the density of clearPoints along the sensing boundary. See Fig. 2 for an illustration. In Fig. 2, clearPoints are marked along the sensing boundary.

Let p_1 present the clearPoint, where the line segment $L(\mathbf{R}, \mathbf{R}^c)$ intersects with the sensing boundary. We enumerate all clearPoints in the clockwise direction starting from p_1 . See Fig. 2.

The SR R lets R^c move back and forth between p_i ($i \in \{1, 2, \dots, B\}$) and R to clean the straight line segment $L(p_i, \mathbf{R})$. For cleaning the open workspace close to R , R^c visits the clearPoints and $\mathbf{R}$ in the following order: $p_1 \rightarrow \mathbf{R} \rightarrow p_2 \rightarrow \mathbf{R} \rightarrow \dots \rightarrow p_B \rightarrow \mathbf{R}$.

Initially, p_1 is selected along the sensing boundary, so that *AttachPoint* exists on the line segment $L(p_1, \mathbf{R})$. Suppose that *AttachPoint* exists on the line segment $L(p_i, \mathbf{R})$, where $i \in \{1, 2, \dots, B\}$. As presented in Fig. 2, the hose expands, so that R^c maneuvers towards p_i .

Let $\mathbf{v}^c(k)$ present the velocity of R^c at sampling-index k . The process model of R^c as it maneuvers towards p_i is

$$\mathbf{R}^c(k+1) = \mathbf{R}^c(k) + \mathbf{v}^c(k) * T, \quad (3)$$

where

$$\mathbf{v}^c(k) = S^c * \frac{\mathbf{p}_i - \mathbf{R}}{\|\mathbf{p}_i - \mathbf{R}\|}. \quad (4)$$

(4) implies that as R^c moves in the direction of $\mathbf{p}_i - \mathbf{R}$ with its maximum speed S^c .

R^c keeps moving in the direction of $\mathbf{p}_i - \mathbf{R}$ until the hose length is maximized, i.e., $\|\mathbf{R}^c - \mathbf{R}\| = H$. Once $\|\mathbf{R}^c - \mathbf{R}\| = H$ is satisfied, R^c gets back to R by shrinking the hose length to zero.

While R^c maneuvers towards p_i , R^c may contact with an obstacle, such as human or L-shaped obstacle in Fig. 2. R^c can use proximity sensors, such as ultrasound sonars, for detection of a close obstacle. Whenever R^c contacts with an obstacle, it gets back to R by shrinking the hose length to zero. Note that tangling of the hose does not happen, according to this shrinking maneuvers.

Once the hose length becomes zero, R rotates by $\frac{2\pi}{B}$, so that *AttachPoint* exists on the line segment $L(p^{i+1}, \mathbf{R})$. During the rotation, R stays stills at its location. After the rotation, the hose expands towards p_{i+1} , so that R^c maneuvers towards p_{i+1} . This iterates while i changes in the following order: $\{1 \rightarrow 2 \rightarrow \dots \rightarrow B\}$. Once R^c finishes rotating by 360 degrees, then this implies that the open workspace close to R has been cleared by R^c .

In Alg. 1, the SR tracks along the shortest path in I to reach f_R . While R maneuvers until reaching f_R , R^c is attached to R at *AttachPoint*. Fig. 3 shows the case where R^c is attached to R (hose length is zero). In this way, R^c can follow R , while R maneuvers until reaching f_R .

4. Simulation results

We verify the outperformance of Alg. 1 through MATLAB simulations. For comparison, we simulate the case where a single large robot vacuum (with high cleaning power) is applied for clearing the workspace. Note that a large robot cannot clear various narrow environments, such as narrow space under a sofa. We show that the proposed tethered robots improve the cleaning efficiency, compared to the case where a single large robot vacuum is applied.

The sensor range r_s is 5 distance units. The hose length H is 7.5 distance units. There are $D = 14$ discretePoints on a sensing boundary. The sampling interval T is 0.1 s.

Recall that R presents the SR with radius r centered at $\mathbf{R} \in \mathcal{R}^2$. In addition, recall that R^c presents the CR with radius r^c centered at $\mathbf{R}^c \in \mathcal{R}^2$. Considering a robot size, we set r as 0.3 distance units, also we set r^c as 0.1 distance unit. This leads to the safety margin, given by $rM = r + 2 * r^c = 0.5$ distance unit. The maximum speed of R is 10 distance units per second.

We set B , the number of clearPoints on a sensing boundary, as 14. We assume that R^c can maneuver under an obstacle, since R^c is a small CR whose role is to clean various environments, such as narrow space under a sofa. Note that R^c can clear a narrow space, which is not reachable by the SR.

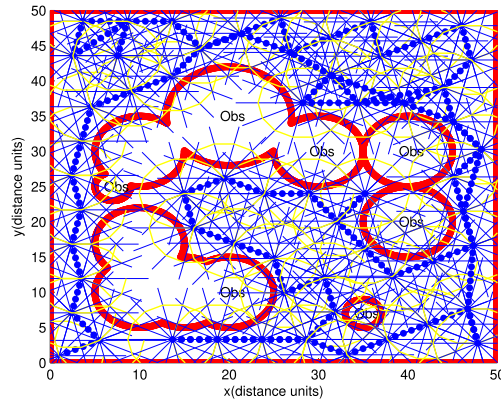


Fig. 4. Scenario 1. The entire sensing boundaries generated by the proposed cleaner system. Obstacles are presented with thick red curves. In order to avoid colliding with obstacles, the SR must maintain at least $rM = r + 2 * r^c$ distance from thick red curves. The sensing boundary of each virtual node is depicted with a yellow circle. The trajectory of R is presented with blue circles. Simple back and forth maneuver of R^c (blue line segments emanating from a virtual node) is applied for clearing the open workspace close to R . R^c can maneuver under an obstacle, since R^c is a small CR whose role is to clean various environments. We acknowledge that due to the limit of hose length H , we may have a space (under an obstacle), which is not cleared by R^c . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.1. Scenario 1

As to Scenario 1, Fig. 4 depicts the entire sensing boundaries built based on Alg. 1. Obstacles are presented with thick red curves. In addition, *Obs* indicates the area associated to obstacles. In order to avoid colliding with obstacles, the SR must maintain at least $rM = r + 2 * r^c$ distance from thick red curves. The sensing boundary of each virtual node is depicted with a yellow circle. The trajectory of R is presented with blue circles. Simple back and forth maneuver of R^c (blue line segments emanating from a virtual node in Fig. 4) is applied for clearing the open workspace close to R .

As to Scenario 1, Fig. 4 depicts the sensing boundaries built by the SR. The initial location of the SR is (1, 1) in distance units. Fig. 4 shows that sensing boundaries of all virtual nodes cover the entire space without obstacles.

Whenever the SR generates a virtual node, the CR R^c cleans the open workspace close to R , as presented in Fig. 2. Simple back and forth maneuver (blue line segments emanating from a virtual node in Fig. 4) is applied for clearing the open workspace close to R . Fig. 4 shows that R^c can maneuver under an obstacle, since R^c is a small CR whose role is to clean various environments, i.e., R^c can clear a narrow space which cannot be reached by the SR.

We assume that the hose length H is not too long, since the SR requires more suction power as the hose length increases. We acknowledge that due to the limit of hose length H , we may have a space (under an obstacle), which is not cleared by R^c . See Fig. 4.

As to Scenario 1, the upper graph of Fig. 5 depicts the XY coordinates of the SR as time (in seconds) elapses, and the lower graph of Fig. 5 depicts the distance between R and its closest obstacle as time (in seconds) elapses. During the clearing process, the SR's distance to its closest obstacle is always bigger than $rM = r + 2 * r^c$.

For comparison, we simulate the case where a single large robot vacuum (with high cleaning power) is applied for clearing the workspace. We show that the proposed tethered robots improve the cleaning efficiency, compared to the case where a single large robot vacuum is applied.

In this comparison, we consider the case where the single large robot maneuvers under Alg. 1, while not using the CR. Assume that the large robot has radius $r = 0.3$ distance units.

As to Scenario 1, Fig. 6 depicts the entire sensing boundaries built by the single large robot. The initial location of the robot is (1, 1) in distance units. The sensing boundary of each virtual node is depicted with a yellow circle. Fig. 6 shows that sensing boundaries of all virtual nodes cover the entire space without obstacles. The trajectory of the single robot is presented with blue circles. The robot cannot maneuver under an obstacle, since the robot is too large.

4.2. Scenario 2

As to Scenario 2, Fig. 7 depicts the entire sensing boundaries generated based on Alg. 1. The initial location of the SR is (1, 1) in distance units. The sensing boundary of each virtual node is depicted with a yellow circle. The trajectory of R is presented with blue circles. Simple back and forth maneuver (blue line segments emanating from a virtual node) is applied for clearing the open workspace close to R . Fig. 7 shows that R^c can maneuver under an obstacle.

As to Scenario 2, the upper graph of Fig. 8 depicts the XY coordinates of the SR as time (in seconds) elapses, and the lower graph of Fig. 8 illustrates the distance between R and its closest obstacle as time (in seconds) elapses. During the clearing process, the SR's distance to its closest obstacle is always bigger than $rM = r + 2 * r^c$.

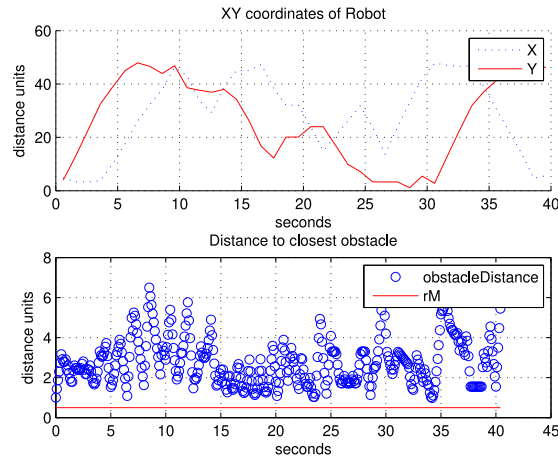


Fig. 5. Scenario 1. The upper graph depicts the XY coordinates of the SR as time (in seconds) elapses, and the lower graph depicts the distance between R and its closest obstacle as time (in seconds) elapses. During the clearing process, the SR's distance to its closest obstacle is always bigger than rM .

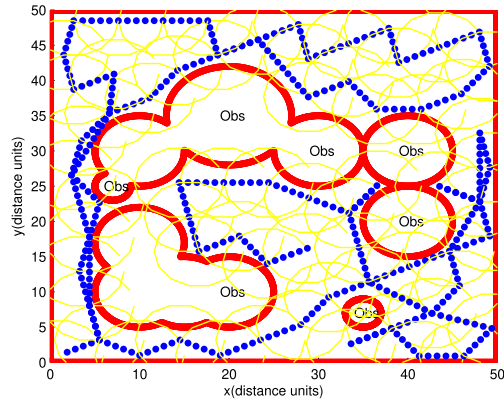


Fig. 6. Scenario 1 using a single large robot. The sensing boundary of each virtual node is presented with a yellow circle. The trajectory of the robot is depicted with blue circles. The robot cannot maneuver under an obstacle, since the robot is too large. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

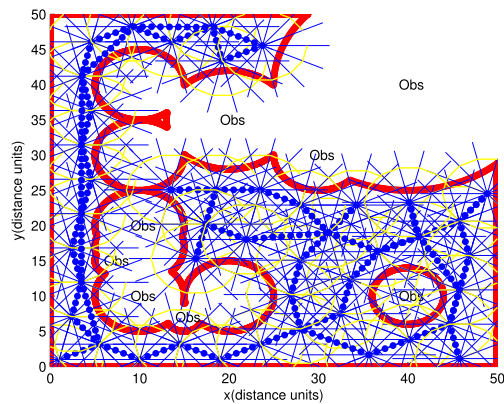


Fig. 7. Scenario 2. The entire sensing boundaries generated by the proposed cleaner system. The sensing boundary of each virtual node is presented with a yellow circle. The trajectory of R is depicted with blue circles. Simple back and forth maneuver (blue line segments emanating from a virtual node) is applied for clearing the open workspace close to R . See that R^c can maneuver under an obstacle. We acknowledge that due to the limit of hose length H , we may have a space (under an obstacle), which is not cleared by R^c . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

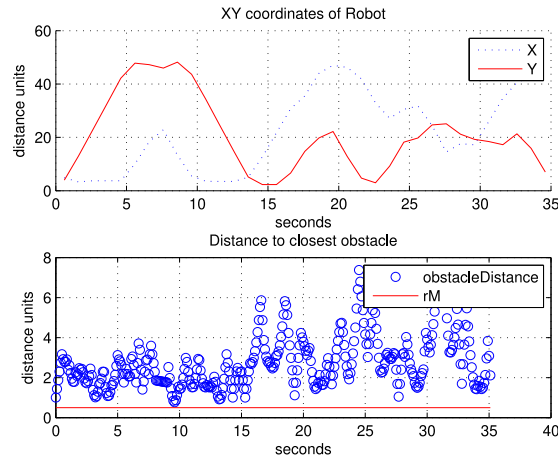


Fig. 8. As to Scenario 2, the upper graph depicts the XY coordinates of the SR as time (in seconds) elapses, and the lower graph illustrates the distance between R and its closest obstacle as time (in seconds) elapses. During the clearing process, the SR's distance to its closest obstacle is always bigger than rM .

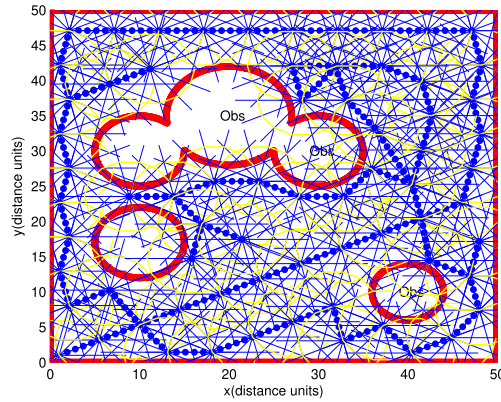


Fig. 9. Scenario 3. The entire sensing boundaries generated by the proposed cleaner system. The sensing boundary of each virtual node is presented with a yellow circle. The trajectory of R is depicted with blue circles. Simple back and forth maneuver (blue line segments emanating from a virtual node) is applied for clearing the open workspace close to R . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.3. Scenario 3

As to Scenario 3, Fig. 9 depicts the entire sensing boundaries generated based on Alg. 1. The initial location of the SR is (1, 1) in distance units. The sensing boundary of each virtual node is depicted with a yellow circle. The trajectory of R is presented with blue circles. Simple back and forth maneuver (blue line segments emanating from a virtual node) is applied for clearing the open workspace close to R . Fig. 9 shows that R^c can maneuver under an obstacle.

As to Scenario 3, the upper graph of Fig. 10 depicts the XY coordinates of the SR as time (in seconds) elapses, and the lower graph of Fig. 10 illustrates the distance between R and its closest obstacle as time (in seconds) elapses. During the clearing process, the SR's distance to its closest obstacle is always bigger than $rM = r + 2 * r^c$.

5. Conclusions

Conventional robot vacuum has a small dust bin, and its suction power is low. To resolve this problem, this article addressed an autonomous robot vacuum system composed of two robots: a small CR and a large SR. The dust sucked by the small CR is transmitted to the large SR through a hose. We address the path plan of these two connected robots, so that the entire open workspace can be cleaned by the CR as fast as possible. Under MATLAB simulations, we demonstrate the effectiveness of path planning for our autonomous robot vacuum system. In the future, we will demonstrate the performance of the proposed dual-robot system utilizing experiments with real robots.

This paper assumes that the large SR is localized in the indoor environment. For robot localization using Lidar, we can use Lidar SLAM in [24,25]. As we do experiments using real robots, we will integrate robot localization for verifying the effectiveness of the proposed dual-robot system rigorously.

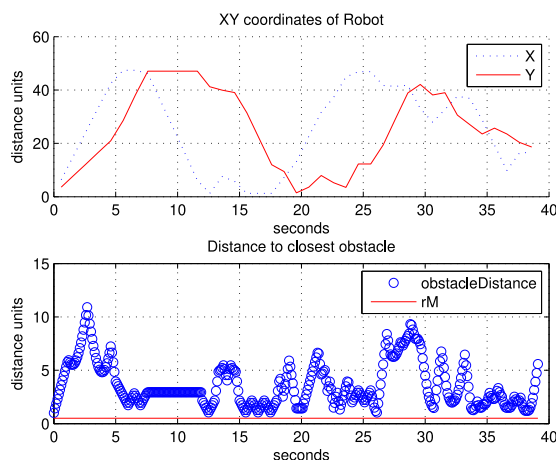


Fig. 10. As to Scenario 3, the upper graph depicts the XY coordinates of the SR as time (in seconds) elapses, and the lower graph illustrates the distance between R and its closest obstacle as time (in seconds) elapses.

An increasing number of robot vacuum cleaners have been hacked previously [27]. The authors of [28] addressed using wifi communication for remote controls of a vacuum robot. The Ref. [27] studied the safety of a robot vacuum cleaner. The authors of [27] attempted to hack a robot vacuum cleaner, and thereby evaluated how secure it is. Inspired by [27,28], we will develop how to control the proposed dual-robot system using wifi communication, such that the system is robust to hacking.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (Grant Number: 2022R1A2C1091682).

References

- [1] T. Asafa, T. Afonja, E. Olaniyan, H. Alade, Development of a vacuum cleaner robot, *Alex. Eng. J.* 57 (4) (2018) 2911–2920.
- [2] M. Caruso, P. Gallina, S. Seriani, On the modelling of tethered mobile robots as redundant manipulators, *Robotics* 10 (2) (2021).
- [3] S. Kim, M. Likhachev, Path planning for a tethered robot using multi-heuristic a with topology-based heuristics, in: 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2015, pp. 4656–4663.
- [4] P. Brass, I. Vigan, N. Xu, Shortest path planning for a tethered robot, *Comput. Geom.* 48 (9) (2015) 732–742.
- [5] R. Teshnizi, D. Shell, Motion planning for a pair of tethered robots, *Auton. Robots* 45 (2021) 693–707.
- [6] E. Galceran, M. Carreras, A survey on coverage path planning for robotics, *Robot. Auton. Syst.* 61 (12) (2013) 1258–1276.
- [7] A. Agarwal, M.-H. Lim, L.C. Woon, A divide and conquer algorithm for rectilinear region coverage, in: IEEE Conference on Robotics, Automation and Mechatronics, 2004, Vol. 2, 2004, pp. 996–1001, <http://dx.doi.org/10.1109/RAMECH.2004.1438055>.
- [8] S. Luo, Y. Singh, H. Yang, J.H. Bae, J.E. Dietz, X. Diao, B.-C. Min, Image processing and model-based spill coverage path planning for unmanned surface vehicles, in: OCEANS 2019 MTS/IEEE SEATTLE, 2019, pp. 1–9, <http://dx.doi.org/10.23919/OCEANS40490.2019.8962662>.
- [9] H. Choset, P. Pignon, Coverage path planning: The boustrophedon cellular decomposition, in: A. Zelinsky (Ed.), *Field and Service Robotics*, Springer London, London, 1998, pp. 203–209.
- [10] D.P. Williams, F. Baralli, M. Micheli, S. Vasoli, Adaptive underwater sonar surveys in the presence of strong currents, in: 2016 IEEE International Conference on Robotics and Automation, ICRA, 2016, pp. 2604–2611.
- [11] C.W. Warren, A technique for autonomous underwater vehicle route planning, *IEEE J. Ocean. Eng.* 15 (3) (1990) 199–204.
- [12] D.P. Williams, On optimal AUV track-spacing for underwater mine detection, in: 2010 IEEE International Conference on Robotics and Automation, 2010, pp. 4755–4762.
- [13] J. Fransman, J. Sijs, H. Dol, E. Theunissen, B.D. Schutter, Distributed constraint optimization for autonomous multi AUV mine counter-measures, in: OCEANS 2018 MTS/IEEE Charleston, 2018, pp. 1–7.
- [14] J. Kim, Multi-robot global sonar survey in the presence of strong currents, *Ocean Eng.* 188 (2019) 106316.
- [15] A. Howard, Multi-robot simultaneous localization and mapping using particle filters, *Int. J. Robot. Res.* 25 (12) (2006) 1243–1256.
- [16] D. Fox, J. Ko, B. Stewart, K. Konolige, B. Limketkai, Distributed multirobot exploration and mapping, *Proc. IEEE* 94 (7) (2006) 1325–1339.
- [17] J. Kim, Topological map building with multiple agents having abilities of dropping indexed markers, *J. Intell. Robot. Syst.* 103 (2021) 1–18.
- [18] J. Kim, Cooperative exploration and networking while preserving collision avoidance, *IEEE Trans. Cybern.* 47 (12) (2017) 4038–4048.
- [19] B. Yamauchi, Frontier-based exploration using multiple robots, in: *Proc. of the Second International Conference on Autonomous Agents*, NY, USA, 1998, pp. 47–53.
- [20] W. Burgard, M. Moors, C. Stachniss, F. Schneider, Coordinated multi-robot exploration, *IEEE Trans. Robot.* 21 (2005) 376–386.

- [21] M. Yim, P. White, M. Park, J. Sastra, Modular self-reconfigurable robots, in: R.A. Meyers (Ed.), *Encyclopedia of Complexity and Systems Science*, Springer New York, New York, NY, 2009, pp. 5618–5631.
- [22] D.M. Hensinger, G.A. Johnston, E.M. Hinman-Sweeney, Self-reconfigurable robots, Tech. Rep., Sandia National Laboratories, 2002.
- [23] M. Yim, W.-m. Shen, B. Salemi, D. Rus, M. Moll, H. Lipson, E. Klavins, G.S. Chirikjian, Modular self-reconfigurable robot systems [grand challenges of robotics], *IEEE Robot. Autom. Mag.* 14 (1) (2007) 43–52.
- [24] W. Hess, D. Kohler, H. Rapp, D. Andor, Real-time loop closure in 2D LIDAR SLAM, in: 2016 IEEE International Conference on Robotics and Automation, ICRA, 2016, pp. 1271–1278.
- [25] J. Zhang, S. Singh, Low-drift and real-time lidar odometry and mapping, *Auton. Robots* 41 (2017) 401–416.
- [26] R. Rodríguez-Puente, M. Lazo-Cortés, Algorithm for shortest path search in geographic information systems by using reduced graphs, *SpringerPlus* 2 (291) (2013) 1–13.
- [27] E. Bröndum, C. Torgilsman, Ethical hacking of a robot vacuum cleaner, Tech. Rep., KTH ROYAL INSTITUTE OF TECHNOLOGY, 2020.
- [28] P.B. Jarande, S.P. Murakar, N.S. Vast, N.P. Ubale, S.S. Saraf, Robotic vacuum cleaner using arduino with wifi, in: 2018 Second International Conference on Inventive Communication and Computational Technologies, ICICCT, 2018, pp. 1513–1517.